



Lista de Exercícios - Assembly ARM
baseada na lista de exercícios preparada pelo Prof. Eduardo dos Santos

- 1) Cem words de 32 bits são armazenados consecutivamente na memória começando no endereço 0x00420A0. Qual é o endereço da palavra que ocupa o 7º espaço e o 22º?
- 2) Supondo que x esteja armazenado em R1, y em R2, z em R3 e o resultado em R0, escreva instruções em linguagem assembly ARM para calcular o resultado das equações abaixo. Considere que x, y e z são inteiros com sinal e que o resultado das expressões possa ser armazenado em 32 bits.
(procure sempre utilizar o menor número possível de instruções)
 - a) $x + y + z$
 - b) $y - x - z$
 - c) $2x + 2y + 2z$
 - d) $5(x + y)$
 - e) $(x + y)(y - z)$
 - f) $3 - 5x - 16y^3z^3$
- 3) Traduza cada um dos seguintes trechos de pseudocódigo em uma sequência de instruções em linguagem assembly ARM. Suponha que as variáveis x e y sejam inteiros com sinal e x esteja em R1 e y esteja em R2.
(procure sempre utilizar o menor número possível de instruções)

a) .

```
if (x == 0)
{
    x = x + 3;
}
```

Não utilize instruções de salto nesta resolução.

b) .

```
if (x >= 5)
{
    x = 0;
}
```

Não utilize instruções de salto nesta resolução.

c) .

```
x = 10;
y = 5;
while (x > 0)
{
    y = y*x;
    x = x - 1;
}
```

d) .

```
if (x < 9)
{
    x = x + 1;
}
else
{
    x = 0;
}
```

Não utilize instruções de salto
nesta resolução.

e) .

```
if (x > 9)
{
    x = 0;
    if (y > 9)
    {
        y = 0
    }
    else
    {
        y = y + 1;
    }
}
else
{
    x = x + 1;
}
```

- 4) Traduza cada um dos seguintes trechos de pseudocódigo em uma sequência de instruções de linguagem assembly ARM. Suponha que x e y sejam inteiros com sinal em R1 e R2, respectivamente.

a) .

```
if ((x == 0x20) || (x == 0x22) || (x == 0x15))
{
    y = 1; // state 0
}
else
{
    y = 0; // state 1
}
```

Não utilize instruções de salto nesta resolução.

b) .

```
if (((x >= 0x10) && (x <= 0x29)) || ((x >= 0x31) && (x <= 0x5c)))
{
    y = 1; // state 0
}
else
{
    y = 0; // state 1
}
```

c) .

```
if (((x >= 0x10) && (x <= 0x49)) || ((x >= 0x51) && (x <= 0x5c)))
{
    y = 1; // state 0
}
else
{
    y = 0; // state 1
}
```

5) Para cada segmento de código assembly determine o valor armazenado em R0 e o estado dos flags N (Negativo), Z (Zero), C (Carry) e V (Overflow) após a execução das instruções.

(procure sempre utilizar o menor número possível de instruções)

a) .
LDR R0, =0x00000000
LDR R1, =0x00000001
ADDS R0, R0, R1

b) .
LDR R0, =0x00000001
LDR R1, =0x00000000
SUBS R0, R0, R1

c) .
LDR R0, =0x80000000
LDR R1, =0x80000001
ADDS R0, R0, R1

d) .
LDR R0, =0x00000000
LDR R1, =0x00000000
SUBS R0, R0, R1

e) .
LDR R0, =0x00000000
LDR R1, =0x00000000
ADDS R0, R0, R1

f) .
LDR R0, =0x80000000
LDR R1, =0x80000000
ADDS R0, R0, R1

g) .
LDR R0, =0x80000000
LDR R1, =0x00000000
SUBS R0, R0, R1

6) Calcule os resultados das seguintes expressões, apresente seus resultados em notação hexadecimal.

- a) $0x\text{CAFE} \ll 2$
- b) $0x\text{BADDCAFE} \gg 24$
- c) $0x\text{BADDCAFE} \ll 16$
- d) $(0x\text{BADDCAFE} \gg 4) \& 0xFF$
- e) $(0x\text{BADDCAFE} \& \sim 0xFF00) | 0x4200$

7) Escreva instruções em assembly para realizar as seguintes operações, considerando os registradores como little-endian (bit 0 é o LSb).
(procure sempre utilizar o menor número possível de instruções)

- a) limpar os bits 4 a 7 de R0
- b) limpar o primeiro e o último bytes de R0
- c) inverter o bit mais significativo de R0
- d) setar os bits 2 a 4 de R0
- e) trocar entre si os bytes mais e menos significativos de R0
- f) substituir os bits 8 a 15 em R0 pelo valor 0x22
- g) $R0 = R1 * 10$ (não use uma instrução de multiplicação)
- h) $R0 = R1 * 100$ (não use uma instrução de multiplicação)
- i) $R0 = R1 / 256$ (não use uma instrução de divisão)
- j) $R0 = R1 \% 256$ (operador mod - resto na divisão)

8) Escreva código em assembly para calcular, em R0, o número de bits "1" em R1. Por exemplo, se $R1 = 0xD0D0CACA$, $R0 = 14$.

9) Se a, b e c são inteiros com sinal de 32 bits armazenados nos endereços de memória 0x40000000, 0x40000004 e 0x40000008 respectivamente, escreva instruções em assembly para calcular:

- a) $a = a + b$
- b) $c = a - b$
- c) $b = a * a$

10) Se uma cadeia de caracteres ASCII (string) terminada em zero for armazenada no endereço de memória 0x40000000, escreva instruções em assembly para:

- a) Contar o número de caracteres na cadeia (sem incluir o zero). Por exemplo, se a cadeia de caracteres ASCII for 0x55 0x54 0x46 0x50 0x52 0x00, seu comprimento será 5.
- b) Armazenar a string em ordem inversa a partir do endereço de memória 0x40001000. Por exemplo, se a string ASCII for 0x55 0x54 0x46 0x50 0x52 0x00, a string reversa armazenada a partir de 0x40001000 será 0x52 0x50 0x46 0x54 0x55 0x00.

11) Se R1 = 0x1000 e R4 = 8, qual o endereço de memória (em hexadecimal) do qual R0 é carregado (lido) e qual é o valor de R1 (em hexadecimal) após a execução de cada uma das instruções da sequência abaixo?

- | | |
|------------------------------|---------------------------|
| a) LDR R0, [R1, #8] | R0=MEM[0x2008], R1=0x1000 |
| b) LDR R0, [R1], #-8 | ? |
| c) LDR R0, [R1, #12]! | ? |
| d) LDR R0, [R1, R4] | ? |
| e) LDR R0, [R1], R4 | ? |
| f) LDR R0, [R1, R4]! | ? |
| g) LDR R0, [R1, R4, LSL #3] | ? |
| h) LDR R0, [R1], R4, LSR #1 | ? |
| i) LDR R0, [R1, R4, LSL #2]! | ? |

12) Dada uma matriz mat no endereço de memória 0x40001000 contendo 64 inteiros de 32 bits mat[0] a mat[63], escreva instruções em assembly correspondentes ao pseudocódigo a seguir. Suponha que i e j sejam inteiros sem sinal de 32 bits armazenados nos endereços de memória 0x40000000 e 0x40000004, respectivamente.

- a) R0 = mat[7]
- b) R0 = mat[i]
- c) i = mat[i] + mat[j]
- d) mat[i] = mat[10] + mat[j]

- 13) Se antes da execução de cada um dos trechos de código abaixo,
 SP = 0x40010000, R4 = 4, R5 = 5 e R6 = 6
 (1) desenhe um diagrama da pilha após a execução das seguintes instruções e
 (2) qual é o conteúdo de R4, R5 e R6 após a execução destes códigos ?

a) .
 PUSH {R4}
 PUSH {R5}
 POP {R6}

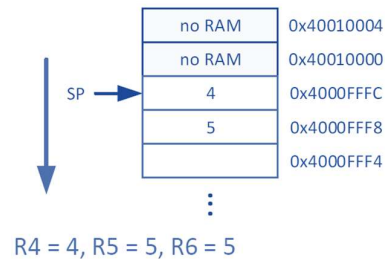
b) .
 PUSH {R4, R5}
 POP {R4, R5}

c) .
 PUSH {R4, R5}
 POP {R5, R4}

d) .
 PUSH {R4, R5, R6}
 POP {R6}
 POP {R5}
 POP {R4}

e) .
 PUSH {R4, R5, R6}
 POP {R6, R4}
 POP {R5}

Resposta da (a)



- 14) Se SP = 0x40040000 e R4 = 0, o que acontece com PC após as instruções?

PUSH {R4}

POP {PC}

- 15) Escreva uma sub-rotina STRLEN que retorne em R0 o comprimento da string ASCII terminada em 0x00. O endereço da string é passado para a sub-rotina em R0.

- 16) Escreva uma sub-rotina LEN que calcule $\sqrt{(x^2+y^2)}$. Suponha que x seja passado para a sub-rotina em R0, y em R1 e que o resultado seja retornado em R0. Considere que x e y são inteiros sem sinal de valor suficientemente baixo para que a soma dos seus quadrados seja inferior à 4G. Suponha também que você possa chamar uma sub-rotina SQRT que retorne a raiz quadrada inteira de R0 em R0.

- 17) A tabela a seguir mostra as pseudo-instruções que são interpretadas pelo assembler Keil armasm. Para as pseudo-instruções: ADRL, LDR, MOV32 e NEG, elabore código que faça uso destas com diferentes argumentos. Selecione bem os argumentos para testar se estas pseudo-instruções são convertidas em sequencias distintas de instruções reais.
- Quais são as sequencias de instruções convertidas ? (analisar com disassembly)
 - Qual a diferença de sintaxe entre a pseudo-instrução LDR e a instrução real LDR?

Dica: use o template de projeto em:

github - dougrenaux_ELEW32_public – asm_minimum

Mnemonic	Brief description	See
ADRL pseudo-instruction	Load program or register-relative address (medium range)	8.3 ADRL pseudo-instruction on page 8-282
CPY pseudo-instruction	Copy	8.4 CPY pseudo-instruction on page 8-284
LDR pseudo-instruction	Load Register pseudo-instruction	8.5 LDR pseudo-instruction on page 8-285
MOV32 pseudo-instruction	Move 32-bit immediate to register	8.6 MOV32 pseudo-instruction on page 8-287
NEG pseudo-instruction	Negate	8.7 NEG pseudo-instruction on page 8-288
UND pseudo-instruction	Generate an architecturally undefined instruction.	8.8 UND pseudo-instruction on page 8-289